Belt-Hook 參數結構 v2 vs v3 對照

> 本文件給已經依 v2 實作的後台 / 整合商，快速對照 v3 變了哪些東西、要注意什麼。完整 v3 規格請見 belt_hook_parameter_protocol.md。

一句話總結

Wire ABI（byte 偏移、欄位寬度、payload 長度）完全相容 v2，但兩個結構各有 byte 重用：

- `hook_settings`（17 bytes）：byte 0 / 1 從 `system_cycle_time` / `advertising_duration` 重用為 `alarm_window_s` / `flags`
- `belt_downlink_header`（24 bytes）：5個欄位中有 2個重用、3個棄用為 reserved padding

兩處的核心精神相同 — wire 位置/寬度不變，但 byte 意義整個換掉。不能視為「同一個欄位改名」。

hook_settings_struct（17 bytes）— byte by byte 對照

Byte	v2 名稱	v2 用途	v3 名稱	v3 用途
0	`system_cycle_time`	系統循環秒數（hook 週期任務的執行間隔）	`alarm_window_s`	警報窗口秒數（1–30）。值 0 是「flag-only write」哨兵 — hook 不更動 settings flash，只處理 `flags` 副作用
1	`advertising_duration`	BLE 廣播持續時間	`flags`	旗標位元組，bit `0x80` = `IN_SAFE_ZONE`（pin counter 0）；其餘 bit 保留必須填 0。transient：hook 寫入 flash 前固定歸 0，不持久化
2	`x.sensing_type`	同 v3	`x.sensing_type`	不變
3–4	`x.sensitivity_high`	同 v3	`x.sensitivity_high`	不變
5–6	`x.sensitivity_low`	同 v3	`x.sensitivity_low`	不變
7–11	`y.*`	同 v3	`y.*`	不變
12–16	`z.*`	同 v3	`z.*`	不變

> v3 端的 `system_cycle_time` / `advertising_duration` 去哪了？ 內化到韌體：hook 端的循環任務間隔與 BLE 廣播時間都由韌體常數決定（mag task = 500 ms、BLE 廣播 always-on 等），不再開放後台調整。如果未來需要重新放出來，會透過完全不同的欄位或新版協定處理，不會再用這兩個 byte。

belt_downlink_header（24 bytes）— byte by byte 對照

Byte	v2 名稱	v2 用途	v3 名稱	v3 用途
0	`reboot_and_ignore_reed`	複合控制：1=reboot；200/201=停用/啟用磁簧	`reboot`	!= 0 → reboot。reed sensor 在 v1 起棄用，magic value 200/201 不再被處理；只剩 reboot 觸發
1	`reset`	重設旗標	`reset`	不變。!= 0 → factory reset（清 NVS + LittleFS）
2–3	`belt_sleep_duration` (LE)	系統循環中的休眠秒數	`power_test_sleep_s` (LE)	重用：> 0 → 一次性 deep sleep N 秒做電流量測，喚醒等同重開機。NOT persisted。v_new always-on 後原本的「循環 sleep 秒數」概念消失
4–5	`uplink_report_interval` (LE)	LoRa 上行週期秒數	`uplink_report_interval_s` (LE)	不變，僅補上 `_s` 單位後綴

6	`beacon_scan_duration`	單次 BLE 掃描秒數	`reserved_a`	棄用。v_new 連續掃描，不再分 cycle
7	`beacon_scan_times`	每循環掃描次數	`beacon_smooth_alpha_x100`	重用：1–100 → BLE-RSSI EWMA α 0.01–1.00，持久化
8–15	`beacon_name_filter[8]`	beacon 名稱前綴過濾（"abeacon"）	`reserved_b[8]`	棄用。v3 改用 UUID 比對，名稱不再參與過濾
16–23	`hook_name_filter[8]`	Hook 名稱前綴過濾（"SQ" / "HOOK-"）	`reserved_c[8]`	棄用。v3 改用 6-byte ID 綁定，名稱只用於人眼辨識

> 重要：byte 7 的解讀換了 — 既有 v2 後台務必檢查。v2 後台若在 byte 7 仍按舊語意填「scans per cycle」（典型 1–10 之間的小整數），v3 belt 會解讀為 EWMA α × 100（例如 5 → α = 0.05，極強平滑），可能造成 RSSI 反應極慢、zone 切換遲緩。升級到 v3 後台時請重設這個 byte，要平常的 α = 0.20 應填 `20`。

外層結構名稱

名稱	v2	v3
Belt header	`belt_downlink_header_v2`	`belt_downlink_header_v3`
完整 payload	`belt_downlink_command_struct_v2`	`belt_downlink_command_struct_v3`
Hook settings 內層	`hook_settings_struct_v2` / `_v2_t`	`hook_settings_struct_v3` / `_v3_t`
Hook BLE 17-byte 寫入	`hook_downlink_command_struct_v2`	`hook_downlink_command_struct_v3`

C 端 typedef name 跟著版次走，舊程式碼直接 `#include "data_types.h"` 編譯時會失敗，強制提示開發者去看 v3 的新 byte 語意，避免「能編過但行為錯」的隱性 bug。

belt_downlink_command_struct_v3（74 bytes payload）— byte 配置

```
byte 0..23   belt header (v3)   24 bytes
byte 24..48  hook[0]            25 bytes (id[6] + reserve[2] + hook_settings[17])
byte 49..73  hook[1]            25 bytes
```

LoRaWAN fPort=30 收到的 payload 長度判斷規則（v2 / v3 共用）：

- 24 bytes → 只更新 belt 自己
- 74 bytes → 更新 belt + 兩支 hook
- 其他長度 → Belt 直接丟棄

v3 新增的兩個功能（v2 後台不會觸發）

1. Flag-only write（單純通知不改 settings）

```
hook_settings.alarm_window_s = 0x00     ← 哨兵
```

hook_settings.flags = 0x80 ← 例：FLAG_IN_SAFE_ZONE

hook_settings.x / y / z = 任意（hook 會忽略）

```
```

收到端	byte 0 = 0 的解讀	byte 1 = 0x80 的解讀
v2 hook	`system_cycle_time = 0`，視為設定值寫入；hook clamp 回 DEFAULT；XYZ 軸全 0 也會被存進 flash → 偵測會壞掉	`advertising_duration = 0x80`，當成 BLE 廣播設定值原樣存入；無實際行為
v3 hook	偵測到 byte 0 = 0 → 走 flag-only path，不寫入 flash；XYZ 全 0 被忽略	bit `0x80` = `IN_SAFE_ZONE` → 觸發 SAFE-zone counter pin

> 結論：v2 hook 收到 v3 後台送的 flag-only write 會把 XYZ 全寫成 0，等同把 hook 偵測弄壞 — 升級必須先燒 hook 再上 v3 後台（見下方升級檢查清單）。反向則 OK：v3 hook 收到 v2 後台原本送的真實 settings（byte 0 = 1–30、byte 1 = 廣播時間值），會 clamp byte 0 到合法範圍並把 byte 1 視為 flags（通常舊值 1 / 2 / 3 等不會湊到 `0x80`，所以不會誤觸 SAFE-zone）。

2. SAFE-zone counter pin（自動 / 手動）

Belt 韌體 v3 在偵測到 SAFE-role beacon 期間 自動 每 3秒對連線中的 hook 各送一次 flag-only write（`alarm_window_s = 0`、`flags = 0x80`），維持 hook 端 10秒的 grace window — counter / status 在 grace window 內被釘在 0，作業員在休息區不會觸發警報。

後台也可以用同樣的 fPort=30 payload 手動 觸發 reset counter（例如排程性的「重置作業日」）：

```
hook[i].id[6]              = 目標 hook MAC
hook[i].reserve            = 0x0000
hook[i].alarm_window_s     = 0x00
hook[i].flags              = 0x80
hook[i].x / y / z          = 全 0
```

Belt 收到後會比對 ID、把 17-byte block 透過 BLE 轉送給對應 hook，hook 走 flag-only path、reset counter。

> v3 的核心設計變化：v2 把 belt → hook 的訊息侷限於「下發設定值」一條路；v3 額外新增「belt 與 hook 透過已連線的 BLE 通道，即時交換 transient 狀態」這條路。flag-only write 是這條路的第一個應用，未來可以在 `flags` 加 bit 擴充（例如 LED 模式、強制喚醒等）而不需要再拓出新欄位。

升級檢查清單

元件	v2 → v3 需要改什麼
Hook 韌體	必須 重新燒錄 v3 (理解 `alarm_window_s = 0` 哨兵 + `flags` bit 0x80；否則 v3 後台送 flag-only write 會打壞 hook 的 XYZ sensitivity)
Belt 韌體	重新燒錄 v3 (含 SAFE-zone 自動觸發 + 新 struct / 欄位名稱)
後台 / LoRa platform — header byte 7	必要：v2 後台若仍按 `beacon_scan_times` 語意填 1–10 之類小整數，v3 belt 會誤判為 EWMA a = 0.01–0.10（強平滑、RSSI 反應慢）。改成填正確的 alpha × 100（典型 20 = a 0.20）
後台 / LoRa platform — 廢棄欄位	把 `beacon_scan_duration`（byte 6）、`beacon_name_filter[8]`（byte 8–15）、`hook_name_filter[8]`（byte 16–23）全部填 0；v3 不再讀，但填入錯誤值不會被偵測
後台 / LoRa platform — hook block	byte 0 / byte 1 重新標示為 `alarm_window_s` / `flags`（值不需變動，但 byte 0 = 0 在 v3 是 flag-only write 哨兵，請避免誤填）

後台 / 想啟用 SAFE-zone 手動 reset	在 hook block 模板上加「flag-only template」(`alarm_window_s=0`, `flags=0x80`) 給操作員選用
後台 payload generator (Python / Node)	把 struct 定義裡的欄位名跟著改；wire bytes 不變所以不必動 generator 邏輯。建議參考 [`belt_admin.py`](../server/simulator/belt_admin.py) 的 v3 包裝寫法

升級順序：先 hook → 再 belt → 最後切換後台到 v3。先燒 belt 而還沒燒 hook 的話，belt 會送 flag-only write 給 v2 hook，造成 hook 的 XYZ sensitivity 被覆寫成 0。

相關文件

- belt_hook_parameter_protocol.md — v3 完整規格 + 範例 payload
- belt_operation.md — Belt 端使用者操作說明（含 SAFE / DANGER zone 行為）
- [`22_XDA003H/docs/hook_operation.md`](../hook/operation.md) — Hook 端使用者操作說明（含 SAFE-zone grace window 描述）